

Hive & Pig: Querying and Cleaning Unstructured Data at Scale

BDA POP — Session 09/22/2026

Why this document exists

Last week's session established that Hadoop's HDFS+YARN pair removes the single-machine ceiling, and that MapReduce is the programming model that runs on top of it — but it also named Hive and Pig only in passing, as tools that “compile down to MapReduce so nobody writes it by hand.” That's true, but it skips the part that actually matters day to day: what does an analyst asking a question, or an engineer cleaning yesterday's raw logs, actually type? This document answers that directly. Hive gives a SQL-shaped front end over data already sitting in HDFS — the same partition-pruning and table-design choices a database administrator makes, applied to a cluster instead of one server. Pig gives a scripting front end for the messier job that has to happen first: turning raw, half-broken order and delivery logs into the clean, structured rows Hive can query in the first place. Both exist because “write a Map function and a Reduce function” is the wrong level of abstraction for either job.

Hive: a SQL front end over data that never leaves HDFS

Hive doesn't store data itself — it sits on top of HDFS (last week's storage layer) and gives it a schema and a query language. Four pieces do the work:

- **Driver** — receives the HiveQL statement, manages the session and the query's lifecycle from submission to result.
- **Compiler** — parses the query, checks it against the schema the Metastore holds, and turns it into an execution plan: historically a MapReduce job, increasingly a Tez or Spark DAG.
- **Metastore** — the piece that makes Hive feel like a database: a separate relational store holding table definitions, column names and types, and which HDFS directory each table's (and each partition's) files actually live in. Hive never touches raw HDFS bytes until a query needs them — schema lookups hit the Metastore, not the data.
- **Execution Engine** — runs the compiled plan against the cluster and streams results back.

This week's example table lives in the Metastore as a single row: `orders`, columns `order_id:int`, `restaurant_id:int`, `city:string`, `order_date:string`, `delivery_time_minutes:int`, partitioned by `order_date`, with its files at `/warehouse/orders/order_date=<date>/`. The `order_date=<date>` segment of that path is the partitioning — each date gets its own HDFS subdirectory, so a query naming a date in its `WHERE` clause can skip every other directory entirely.

Managed vs. external tables

Kind	Hive owns the data?	Typical use
Managed (internal)	Yes — <code>DROP TABLE</code> deletes the HDFS files too	Data Hive itself created or fully controls
External	No — Hive only owns the schema/metadata	Raw files another pipeline (Flume, Sqoop) already writes to HDFS

Getting this choice wrong is a common production mistake: pointing a *managed* table at a shared raw log directory means one careless `DROP TABLE` in Hive deletes files other pipelines still depend on. External tables exist specifically so Hive can query data it doesn't own the lifecycle of.

Partitioning and bucketing

Partitioning splits a table's files by a column's value into separate HDFS directories, so a query filtering on that column reads only the matching directories. **Bucketing** goes further *within* a partition: it hashes a column into a fixed number of files, so joins and sampling on that column don't need to scan the whole partition either.

This week's simulated order-log table holds 5,401 rows spanning 14 days across 6 cities. A query naming one specific day in its `WHERE` clause, against a partitioned table, reads only that day's directory — 361 rows — instead of scanning all 5,401. That's roughly **15x** fewer rows read for a question that only ever needed one day's answer. Bucketing `restaurant_id` into 8 buckets, separately, spreads those same rows close to evenly (each bucket held between 626 and 718 rows in this run) — even enough that a join or a sample on `restaurant_id` touches a predictable, bounded slice of each partition rather than all of it.

Metric	Value	Threshold	Status
Naive full-table scan vs. one partitioned query (rows read)	5,401 vs. 361 (~15x more)	should be $\approx$ 1 partition's worth	Flagged
Bucket row-count spread across 8 buckets	626–718 ($\approx$ 13% spread)	roughly even across buckets	Within threshold

Reading the panel: naming the right column in `WHERE` isn't a style preference in Hive — it's the difference between reading a fifteenth of the table and reading all of it, on data that was always going to live in HDFS either way.

HiveQL in practice

A representative query for this table:

```
SELECT city, COUNT(*) AS n_orders, AVG(delivery_time_minutes) AS avg_delivery
FROM orders
WHERE order_date = '<target_date>'
GROUP BY city;
```

Run against one day's raw partition, this returns a per-city order count and average delivery time — but `AVG()` over a column that still contains missing or negative values gives a quietly wrong answer, or forces every query that touches the column to add a defensive `WHERE delivery_time_minutes IS NOT NULL`. Hive *can* query dirty data. It shouldn't have to.

The data-quality problem Hive alone doesn't solve

Of the 5,401 simulated order-log rows, 216 (4%) carry a missing or negative `delivery_time_minutes` — a deliberately injected slice standing in for what a raw ingestion pipeline actually hands a cluster before anyone cleans it.

Metric	Value	Threshold	Status
Raw log rows with invalid <code>delivery_time_minutes</code>	216 / 5,401 (4%)	0% (every row should be usable)	Flagged

Reading the panel: every `AVG(delivery_time_minutes)` query in the previous section is already quietly off by a few percent of its input, and a query without the defensive `IS NOT NULL` filter would be worse — a -1

value silently pulls a city's average down with no error raised anywhere. This needs a cleaning step *before* Hive, not a workaround inside every query written against it.

Pig: a scripting front end for the cleanup pass

Hive is built for someone who already knows what question they want answered. Pig is built for someone who needs to describe a multi-step transformation — load, filter, group, join, store — as a short, readable script, without hand-writing MapReduce for each step. Pig Latin (Pig's language) reads like a pipeline, not a single query, which is exactly the shape “turn today's raw order log into a clean table” takes:

```
raw_orders = LOAD 'orders_raw.csv' USING PigStorage(',')
            AS (order_id:int, restaurant_id:int, city:chararray,
               order_date:chararray, delivery_time_minutes:int);
clean_orders = FILTER raw_orders BY delivery_time_minutes IS NOT NULL
              AND delivery_time_minutes > 0;
by_city      = GROUP clean_orders BY city;
city_stats   = FOREACH by_city GENERATE
              group AS city, COUNT(clean_orders) AS n_orders,
              AVG(clean_orders.delivery_time_minutes) AS avg_delivery;
STORE clean_orders INTO '/warehouse/orders_clean' USING PigStorage(',');
```

Applying the same `FILTER` logic to this week's simulated data drops exactly the 216 dirty rows, leaving 5,185 clean rows with zero remaining invalid values:

Metric	Value	Threshold	Status
Invalid delivery_time_minutes after the Pig-style FILTER	0% (0 / 5,185)	0%	Within threshold

Reading the panel: the `FILTER` step is Pig's version of last week's diagnostic pattern — it doesn't make the raw data less messy where it landed, it removes the mess before anything downstream (Hive, in this pipeline) ever queries it. `clean_orders` is what would actually get `STORED` into `/warehouse/orders_clean`, the directory a *managed* Hive table would then be pointed at — closing the loop back to the Hive query from the previous section, now running against trustworthy data.

Hive vs. Pig: same engine underneath, different job on top

Aspect	Hive	Pig
Language shape	Declarative, SQL-like (HiveQL)	Procedural, step-by-step (Pig Latin)
Written by	Analysts who know SQL	Data engineers building pipelines
Best fit	“What's the answer to this question?”	“Turn this raw mess into that clean table.”

Both still compile down to the same batch execution model (MapReduce, increasingly Tez) that last week's session covered — the difference is entirely in who's writing the logic and what shape their problem takes, not in what runs on the cluster underneath.

The rest of the ecosystem: getting data in and jobs coordinated

Hive and Pig both assume the data is already in HDFS and that something runs their jobs on a schedule. Four more tools cover exactly those gaps:

Tool	Provides	Reached for when
Sqoop	Bulk transfer between HDFS and relational databases	Nightly-import last month's orders table from a production MySQL database
Flume	Streaming ingestion of continuous log/event data into HDFS	Continuously stream app/server logs into HDFS as they're generated
Oozie	Workflow scheduling that chains jobs together	"Every night at 2am: pull new orders, clean with Pig, then refresh the Hive table"
ZooKeeper	Distributed coordination — leader election, shared config, locking	Multiple cluster services need to agree on who's currently in charge

Put together, a realistic nightly pipeline reads: **Sqoop/Flume** land raw data in HDFS → **Pig** cleans and reshapes it → a Hive **managed table** exposes the clean result for querying → **Oozie** is what actually ran all three steps in order, unattended, every night → **ZooKeeper** keeps the cluster's own services agreeing on which instance is currently active. None of these compete with Hive or Pig — each fills a gap neither one was built to cover.

Summary

Check	Result
Naive full-table scan vs. partition-pruned query (rows read)	Flagged — ~15x more rows without partitioning
Bucket row-count spread across 8 buckets	Within threshold — 626–718 rows, roughly even
Raw order-log rows with invalid delivery_time_minutes	Flagged — 4% (216 / 5,401) before cleaning
Same check after the Pig-style FILTER step	Within threshold — 0% (0 / 5,185)

Hive's value is a schema and a query language over data that was always going to live in HDFS; Pig's value is a readable pipeline for the transformation that has to happen before that data is trustworthy enough to query. Neither replaces MapReduce — both are what most people actually touch instead of it.